

Financial Mathematics Calculator

Mvelo Mnengi

North-West University
June 14, 2026

Abstract

This document describes a Python-based financial calculator developed as a project for BWIA121 (Introduction to Actuarial Sciences) at North-West University. The program implements standard time value of money calculations including interest rate conversions, annuity valuations, loan amortization components, and basic graphing capabilities. The calculator accepts user inputs through a text-based menu system and returns formatted numerical results.

1 Introduction

The calculator performs eight categories of financial calculations commonly encountered in introductory actuarial science:

1. Interest rate conversions
2. Annuity calculations (level payments)
3. Interest and capital component analysis
4. Increasing annuities (geometric growth)
5. Simple interest
6. Effective interest
7. Effective periodic interest
8. Continuous interest (force of interest)
9. Basic graphical visualization

2 Mathematical Framework

2.1 Basic Time Value of Money

The fundamental relationship between present value (PV) and future value (FV) with compound interest is:

$$FV = PV(1 + i)^n$$

where i is the effective interest rate per period and n is the number of periods.

2.2 Interest Rate Conversions

The calculator converts between five interest rate types:

2.2.1 Simple Interest

$$A = P(1 + rt)$$

2.2.2 Effective Interest Rate

The annual effective rate i satisfies $FV = PV(1 + i)^n$.

2.2.3 Nominal Interest Rate

A nominal rate $i^{(m)}$ compounded m times per year relates to the effective rate by:

$$1 + i = \left(1 + \frac{i^{(m)}}{m}\right)^m$$

2.2.4 Effective Periodic Rate

The rate per compounding period i_{period} is:

$$i_{period} = (1 + i)^{1/m} - 1$$

2.2.5 Continuous Interest (Force of Interest)

The force of interest δ is defined as:

$$\delta = \ln(1 + i)$$

$$FV = PV \cdot e^{\delta t}$$

2.3 Annuity Formulas

2.3.1 Ordinary Annuity (Payments in Arrears)

Future value:

$$FV = X \cdot \frac{(1 + i)^n - 1}{i}, \quad i \neq 0$$

$$FV = X \cdot n, \quad i = 0$$

Present value:

$$PV = X \cdot \frac{1 - (1 + i)^{-n}}{i}, \quad i \neq 0$$

$$PV = X \cdot n, \quad i = 0$$

2.3.2 Annuity Due (Payments in Advance)

Future value:

$$FV = X(1 + i) \cdot \frac{(1 + i)^n - 1}{i}$$

Present value:

$$PV = X(1 + i) \cdot \frac{1 - (1 + i)^{-n}}{i}$$

2.4 Loan Amortization

For a loan of amount L with periodic payment P at interest rate i per period:

The balance outstanding after k payments:

$$B_k = L(1+i)^k - P \cdot \frac{(1+i)^k - 1}{i}$$

Interest component of payment k :

$$I_k = B_{k-1} \cdot i$$

Capital component:

$$C_k = P - I_k$$

2.5 Geometrically Increasing Annuities

For payments increasing at rate j per period:

$$FV = X \cdot \frac{(1+i)^n - (1+j)^n}{i-j}, \quad i \neq j$$

$$FV = X \cdot n \cdot (1+i)^{n-1}, \quad i = j$$

3 Program Structure

3.1 Input Validation

The program implements four input validation functions:

```
1 get_int_input(prompt)           # Returns integer value
2 get_float_input(prompt)         # Returns float value
3 get_positive_float_input(prompt) # Returns float > 0
4 get_non_negative_float_input(prompt) # Returns float >= 0
5 get_positive_rate_input(prompt) # Returns rate >= 0
```

Listing 1: Input Functions

Each function loops until the user provides a valid input of the required type.

3.2 Menu System

The main menu presents ten options. The program remains in a loop until the user selects option 10 (Exit). Each selection calls the appropriate function or sub-menu.

3.3 Calculation Functions

Each calculation type is implemented in its own function. For example, the future value of an ordinary annuity:

```

1 def annuity_future_value():
2     print("\nFuture value of a level annuity (arrears).")
3     initial_amount = get_positive_float_input("Enter the periodic
4     payment amount (X): ")
5     rate = get_positive_rate_input("Enter the effective interest rate
6     per period (%): ") / 100
7     time = get_positive_float_input("Enter the time in years: ")
8     periods = get_positive_float_input("Enter the number of compounding
9     periods per year: ")
10    number_of_payments = periods * time
11
12    if rate == 0:
13        future_value = initial_amount * number_of_payments
14    else:
15        future_value = initial_amount * (((1 + rate) **
16        number_of_payments - 1) / rate)
17    print(f"The future value is: R{future_value:.2f}")

```

Listing 2: Annuity Future Value Function

3.4 Graphing Function

The program includes three graph types using matplotlib:

1. Future value comparison (arrears vs advance)
2. Present value comparison (arrears vs advance)
3. Loan amortization (stacked bar chart of interest vs capital components)

All graphs are generated using user-provided input values rather than pre-loaded sample data.

4 Error Handling

The calculator handles several edge cases:

4.1 Zero Interest Rate

When the interest rate is zero, annuity formulas divide by zero. The program detects this and uses linear formulas instead.

4.2 Logarithm Domain Errors

Functions that solve for time or number of payments use logarithms. The program validates that the logarithm argument is positive before calling `ln()`.

4.3 Insufficient Loan Payments

When a user enters a payment amount too small to ever repay a loan, the program displays an error message instead of attempting the calculation.

4.4 Negative Balances

After calculations that could produce negative balances (e.g., overpaying a loan), the program clamps the result to zero.

5 Limitations

The calculator has three main limitations. First, interest rates are assumed constant over the entire calculation period; variable rates are not supported. Second, only geometric payment increases are implemented; arithmetic gradient annuities are not included. Third, the graphical output is limited to basic line and bar charts without interactive features or data export capabilities.

6 Example Calculations

6.1 Example 1: Future Value of an Annuity

Inputs:

- Payment: R1,000 per month
- Rate: 6% per annum (0.5% per month)
- Time: 5 years
- Periods: 12 per year

Output: Future value = R69,770.00

6.2 Example 2: Loan Payment Calculation

Inputs:

- Loan amount: R200,000
- Rate: 9% per annum (0.75% per month)
- Time: 4 years (48 months)

Output: Monthly payment = R4,977.47

6.3 Example 3: Interest Rate Conversion

Inputs:

- Nominal rate: 10%
- Given periods: 2 (semi-annually)

Output: Effective rate = 10.25%

7 Technical Requirements

The program requires:

- Python 3.x
- matplotlib library ('pip install matplotlib')

8 Conclusion

This calculator implements standard financial mathematics formulas for interest conversions, annuities, loan amortization, and increasing payment streams. The program demonstrates proficiency in the fundamentals of actuarial and financial mathematics through correct application of time value of money principles and appropriate handling of edge cases such as zero interest rates.